

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
.Net Core and Progressive Web Application Practical

Practical – V(Curiosity Question)

Roll No.: T005	Name: Ritesh Chaubey
Class:TYIT	Batch:1
Date of Assignment: 21/07/2026	Date/Time of Submission: 28/07/2026

Here is a clearer and more professional version of your algorithm:

ALGORITHM

1. Open Visual Studio.
2. Create a new ASP.NET Core Web App (Model-View-Controller) project.
3. Install the following required NuGet packages:
 - Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
 - Microsoft.EntityFrameworkCore.Design
 - Microsoft.VisualStudio.Web.CodeGeneration.Design
4. Create a Product model with the following properties:
 - Id
 - Name
 - Category
 - Price
 - Quantity
5. Create an ApplicationDbContext class that inherits from **DbContext** and add a **DbSet<Product>** property.
6. Configure the SQL Server connection string in the **appsettings.json** file.
7. Register the **ApplicationDbContext** in **Program.cs** using the **AddDbContext()** method.

Open the Package Manager Console and execute the following commands to create and apply the database migration:

Add-Migration InitialCreate

Update-Database

- 8.
9. In Solution Explorer, right-click the Controllers folder and select Add → New Scaffolded Item.
10. Choose MVC Controller with views, using Entity Framework.
11. Select the Product model class and ApplicationDbContext, then click Add to generate the controller and CRUD views automatically.
12. Build and run the application.
13. Navigate to the Products page from the application.
14. Click Create New and enter the product details.
15. Save the record and verify that the new product appears in the product list.
16. Use the Edit option to modify the details of an existing product and save the changes.
17. Use the Details option to view the complete information of a selected product.
18. Use the Delete option to remove a product from the database and confirm the deletion.
19. Verify that all CRUD (Create, Read, Update, and Delete) operations are functioning correctly and that the database is updated accordingly.
20. Close the application and stop debugging.

This version is more structured, uses consistent terminology, and clearly explains each step while maintaining the correct sequence of the CRUD implementation process.

CODE:

(ProductsController.cs):

```
@model IEnumerable<Practical_5_CQ.Models.Product>

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <a asp-action="Create">Create New</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Name)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Category)
            </th>
            <th>
```

```

        @Html.DisplayNameFor(model => model.Price)
    </th>
    <th>
        @Html.DisplayNameFor(model => model.Quantity)
    </th>
</th></th>
</tr>
</thead>
<tbody>
@foreach (var item in Model) {
    <tr>
        <td>
            @Html.DisplayFor(model => item.Name)
        </td>
        <td>
            @Html.DisplayFor(model => item.Category)
        </td>
        <td>
            @Html.DisplayFor(model => item.Price)
        </td>
        <td>
            @Html.DisplayFor(model => item.Quantity)
        </td>
        <td>
            <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
            <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
            <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
        </td>
    </tr>
}
</tbody>
</table>

```

(Program.cs):

```

using Microsoft.EntityFrameworkCore;
using Pratical_5_CQ.Data;
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios,
    see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

```

```

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();

```

(appsetting.json):

```

{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=(localdb)\\MSSQLLocalDB;Database=ShoppingDB;Trusted_Connection=True;TrustServerCertificate=True;"
    "AppDbContext":
      "Server=(localdb)\\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

(AppDbContext.cs):

```

using Microsoft.EntityFrameworkCore;
using Pratical_5_CQ.Models;
namespace Pratical_5_CQ.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }
    }
}

```

(Product.cs):

```

namespace Pratical_5_CQ.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Category { get; set; }
        public decimal Price { get; set; }
        public int Quantity { get; set; }
    }
}

```

Index

[Create New](#)

Name	Category	Price	Quantity	
Laptop	Electronics	65000.00	5	Edit Details Delete
Mobile	Electronics	25000.00	10	Edit Details Delete
BackPack	Bag	1800.00	14	Edit Details Delete
Shoes	Footwear	3400.00	10	Edit Details Delete

Edit Product

Name

Category

Price

Quantity

[Save](#)

[Back to List](#)

Edit Product

Name

Category

Price

Quantity

[Back to List](#)

Index

[Create New](#)

Name	Category	Price	Quantity	
Laptop	Electronics	65000.00	5	Edit Details Delete
Printer	Electronics	9500.00	10	Edit Details Delete
BackPack	Bag	1800.00	14	Edit Details Delete
Shoes	Footwear	3400.00	10	Edit Details Delete